

COMP3320 Week 1, Lecture 2

Rhys Hawkins

Semester 2, 2024

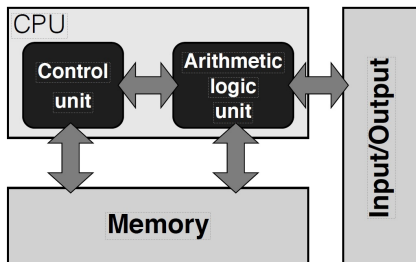
Last Lecture: Errata/Update

- Labs

Lab	Time	Location
1	Wed 12:00 - 13:30	CSIT Building 108 N109
2	Thu 14:00 - 15:30	CSIT Building 108 N109

- Lecture Recordings

Last Lecture



- von Neumann Architecture
- Instructions are stored in memory
- von Neumann bottleneck
- Inherently sequential architecture: single instruction, single data (SISD)

Last Lecture

Admin

- Course Representative(s)
- Create NCI Account
- Labs start next week
- Assignment introduced next week

Plan

This week

- Scientific Code Life-cycle
- Defining/Measuring Time
- Counting FLOPs
- Profiling

Next week

- Floating Point/Precision
- Peak Performance
- Assignment Background

Scientific Code Development

Scientific codes are invariably complex

- First: prototype code and demonstrate correctness
 - Can initially be coded in a rapid prototyping language such as Python or Julia
 - Run at desktop scale and make predictions about scaling to relevant problem sizes
- Second: develop code using HPC paradigms
 - Update code to run using OpenMP, OpenMPI, CUDA etc for parallel efficiency and scale.
 - Generate series of benchmarks and validation tests.
- Finally: analyse performance, tune code, scale to thousands of cores
 - find and measure critical parts of code
 - determine suitable optimizations and implement them
 - measure improvement and verify correctness (and repeat)

Scientific Code Development

Hardware is ever changing

- Performance of CPUs changes between generations
- New instructions are introduced regularly (MMX, SSE, AVX etc)
- Different paradigms: Vector machines, Large CPU clusters, GPGPU (CUDA, HIP, OpenCL etc), Heterogeneous Computing

Understanding and measurement of code performance is challenging.

- Code analysis, $O(n)$ and scaling
- Operating system/Manual Measurement
- Hardware counters

Counting Floating Point Operations

- FLOPs measure the complexity of computation
- 1 FLOP is a basic unit
 - 1 addition, subtraction, multiplication or division
- A **crude** measure of algorithm expense
- CPUs have advertised theoretical maximum number of FLOP/s

What we generally care about is scaling

- A problem size is usually defined by some number n , e.g. n by n grid, or n molecules
- FLOPs can be expressed as a function of n , e.g. $(2n - 1)$
- FLOPs scale as $O(n)$

Usually only the highest order is important

- e.g. for $n \times n$ matrix $\mathbf{B}$ and length n vectors $\mathbf{c}, \mathbf{d}$
- $\mathbf{Bc} + \mathbf{d} \equiv 2n^2 + 2n \text{ FLOPs} \rightarrow O(n^2)$

Floating Point Operations: Examples

```
int i;  
float A[N], B[N]  
float s = 0.0;  
for (i = 0; i < N; i++) {  
    s = s + A[i] * B[i];  
}
```

- Vector dot product
- N multiplications and N additions
- $O(N)$

Floating Point Operations: Examples

```
int i;
float A[M][N], V[N], W[N]
float s;
for (j = 0; j < N; j++) {
    s = 0.0;
    for (i = 0; i < N; i++) {
        s = s + A[j][i] * V[i];
    }
    W[j] = s;
}
```

- Dense Matrix-Vector Multiplication
- N^2 multiplications and N^2 additions
- $O(N^2)$

Floating Point Operations: Examples

```
int i;
float V[N], W[N]
float s;
for (j = 0; j < N; j++) {
    s = 0.0;
    for (i = j + 1; i < N; i++) {
        float dv = V[i] - V[j];
        s = s + dv*dv;
    }
    W[j] = s;
}
```

- A common pattern in nested for loops (note the inner loop depends on the outer loop variable j)
- One of each of add, subtract and multiply in the inner loop
- Inner loop called $N(N - 1)/2$ times.
- Scaling is still $O(N^2)$

Counting Floating Point Operations

Limitations

- A useful first order approximation
- Floating point operations are a part of the performance of a piece of scientific software
- Many scientific codes are memory bound which severely impacts performance as problem size increases (more on this later)
- Only tells you about scaling, still need a measurement.

Performance Measurement

People begin to consider performance when:

- Their workload has increased and they need to squash 25 hours of computing into 24 hours
- The workload of others using the same computing resource has increased
- They are considering the purchase of a new computer and need to assess its performance
- They need to understand the computational requirements at scale to request time on a super computer
- Need to analyse larger models or more of them

First we need to know the meaning of time and how to measure it

Defining Time

Wall Clock Time

Elapsed time - our perceived time

Process Time

Resource time used up by the CPU

Defining Time

- $t_s \equiv$ Time slice interval
 - Number of clock ticks or system interrupts = $\frac{1}{t_s}$
 - For process time an interval timer acts as the system clock
 - Typically $t_s \sim 0.01\text{s}$ which means 100 interrupts/second
- $t_R \equiv$ Timer resolution
 - How accurately can you measure time
 - **Note:** Low t_R means high time resolution
 - $t_R = t_s$ for process time
- $t_C \equiv$ Timer call overhead
 - How long calling the timer takes

Measuring Time - Reliability Issues

What value of t_i is appropriate?

	Wall	Process
Timer Resolution	high	low (t_s)
Timer Call Overhead	low	high
Effect of Interrupts	high	low
Appropriate Timing Interval	$< t_s$	$> 100t_s$

- Timing Error is bounded by $2t_R + t_C$

Scales of Timings

- Machine clock cycle time
 - 1GHz clock equivalent to 1 ns
- Time for basic floating point operations: eg addition/multiplication
 - Multiple clock cycles
- Critical inner loops
 - How to identify these
- Whole Applications
 - Seconds/Minutes/Hours

Can all of these operations be timed?

Total Program Timing

- Unix shells provide the `time` utility, e.g.

```
me$computername> time ./myprogram  
real    2.98  
user    1.70  
sys     0.18
```

- Process time = user + sys
- High system time may indicate memory paging and/or I/O
- Ratio of user + sys time to elapsed time can reflect other users on the machine
- Under `csh`, we get information on average memory usage, I/O operations, page faults and swaps

Manual Timing: Process Time

```
#include <time.h>
#include <sys/times.h>
#include <unistd.h>
#include <sys/time.h>

int main(int argc, char *argv[]) {
    struct tms start, end;
    long tick
    tick = sysconf(_SC_CLK_TCK);
    printf("Ticks per second %ld\n", tick);
```

```
    times(&start);
```

```
    do_something();
```

```
    times(&end);
```

```
    printf("Start: %ld %ld\n", start.tms_utime, start.tms_stime);
```

```
    printf("  End: %ld %ld\n", end.tms_utime, start.tms_stime);
```

```
    return 0;
```

```
}
```

Manual Timing: Process Time

```
#include <time.h>
#include <sys/times.h>
#include <unistd.h>
#include <sys/time.h>

int main(int argc, char *argv[]) {
    struct tms start, end;
    long tick
    tick = sysconf(_SC_CLK_TCK);
    printf("Ticks per second %ld\n", tick);

    times(&start);

    do_something();

    times(&end);

    printf("User: %f\n", (float)(end.tms_utime - start.tms_utime) / (float)(tick));
    printf(" Sys: %f\n", (float)(end.tms_stime - start.tms_stime) / (float)(tick));

    return 0;
}
```

Manual Timing: Wall Time

```
#include <time.h>
#include <sys/times.h>
#include <unistd.h>
#include <sys/time.h>

int main(int argc, char *argv[])
{
    struct timeval start, end;

    gettimeofday(&start, NULL);

    do_something();

    gettimeofday(&end, NULL);

    printf("Start: %ld %ld\n", start.tv_sec, start.tv_usec);
    printf("  End: %ld %ld\n", end.tv_sec, end.tv_usec);
}
```

Manual Timing: Wall Time

```
#include <time.h>
#include <sys/times.h>
#include <unistd.h>
#include <sys/time.h>

int main(int argc, char *argv[])
{
    struct timeval start, end;
    float elapsed;

    gettimeofday(&start, NULL);

    do_something();

    gettimeofday(&end, NULL);

    elapsed = (float)(end.tv_sec - start.tv_sec) +
              (float)(end.tv_usec - start.tv_usec)/1.0e6;

    printf("Elapsed: %f\n", elapsed);
}
```

Manual Timing: Hardware Counters

```
#include <papi.h>
int main(int argc, char *argv[])
{
    PAPI_library_init(PAPI_VER_CURRENT);

    const int NUM_EVENTS = 1;
    int Events[NUM_EVENTS] = {PAPI_TOT_CYC};
    long long values[NUM_EVENTS];

    int EventSet = PAPI_NULL;
    PAPI_create_eventset(&EventSet);
    PAPI_add_events(EventSet, Events, NUM_EVENTS);

    PAPI_start(EventSet);

    do_something();

    PAPI_stop(EventSet);

    printf("Measured %lld cycles\n", values[0]);
    return 0;
}
```

Manual Timing: Issues

- Resolution (and overhead): You should have some idea of its value
 - In some cases it may not be what is reported in a man page, eg it may say microseconds ($1e-6$) but are all the digits meaningful
- CPU Time: Take care with the meaning of CPU time. Some timing routines switch from CPU to elapsed time if the program is running in parallel
- Baseline: Timing provides a baseline from which to judge performance tuning or comparative machine performance
- Placement: How do we know where to place timing calls!
 - Unix provides us with a number of tools to help with this (profiling)

Floating Point Operations: Hardware Counters

```
#include <papi.h>
int main(int argc, char *argv[])
{
    PAPI_library_init(PAPI_VER_CURRENT);

    const int NUM_EVENTS = 2;
    int Events[NUM_EVENTS] = {PAPI_TOT_CYC, PAPI_SP_OPS};
    long long values[NUM_EVENTS];

    int EventSet = PAPI_NULL;
    PAPI_create_eventset(&EventSet);
    PAPI_add_events(EventSet, Events, NUM_EVENTS);

    PAPI_start(EventSet);

    do_something();

    PAPI_stop(EventSet);

    printf("Measured %lld cycles\n", values[0]);
    printf("Measured %lld FLOPs\n", values[1]);
    return 0;
}
```

Floating Point Operations: Hardware Counters

```
#include<papi.h>
int main(int argc, char *argv[])
{
    PAPI_library_init(PAPI_VER_CURRENT);

    const int NUM_EVENTS = 2;
    int Events[NUM_EVENTS] = {PAPI_TOT_CYC, PAPI_SP_OPS};
    long long values[NUM_EVENTS], start, stop;

    int EventSet = PAPI_NULL;
    PAPI_create_eventset(&EventSet);
    PAPI_add_events(EventSet, Events, NUM_EVENTS);

    PAPI_start(EventSet)

    start = PAPI_get_virt_usec();

    do_something();

    stop = PAPI_get_virt_usec();

    PAPI_stop(EventSet, values);

    printf("Measured %lld cycles\n", values[0]);
    printf("Measured %lld FLOPs\n", values[1]);
    printf("Measured %lld MFLOPs\n", values[1]/(stop - start));

    return 0;
}
```

Summary

- Manual timing possible by instrumenting the code
 - Counting time slices
 - 10ms accuracy
 - Better for longer timing (second or more)
 - Wall time
 - Uses high resolution system clocks
 - Measured elapsed time, not direct time of your code
 - Better for shorter lengths, ideally less than 10ms
 - Adversely affected by system load
 - PAPI/Performance Counters
 - Lots of potentially useful information (beyond just timing)
 - Hardware/OS dependent
- Complexity of CPUs/code means that timing is our best method of measuring effect of optimisations!
- Floating Point Operations
 - Code analysis (big O notation) and scalability
 - Performance Counters (Hardware/OS dependent)